SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1702**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 50%

QUESTIONS: 5 **Duration : 1 Hour**
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I M.Santosh(192521007) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

Annexure A

Analytical Problem Solving

1. Solve the Water Jug problem: you are given 2 jugs, a 4-gallon one and 3-gallon one. Neither has any measuring maker on it. There is a pump that can be used to fill the jugs with water. How can you get exactly 2 gallons of water into 4-gallon jug? Explicit assumptions: A jug can be filled from the pump, water can be poured out of a jug onto the ground, water can be poured from one jug to another and that there are no other measuring devices available.

Answer:

Formulation: State = (x, y), where x = gallons in the 4-gallon jug (0–4) and y = gallons in the 3-gallon jug (0–3). Initial state (0,0). Goal state (2, y) for any y.

Production rules: (1) Fill 4-gal: $(x,y)\rightarrow(4,y)$ if $x<4$. (2) Fill 3-gal: $(x,y)\rightarrow(x,3)$ if $y<3$. (3) Empty 4-gal: $(x,y)\rightarrow(0,y)$ if $x>0$. (4) Empty 3-gal: $(x,y)\rightarrow(x,0)$ if $y>0$. (5) Pour 3-gal into 4-gal until full: $(x,y)\rightarrow(4, y-(4-x))$ if $x+y\geq4$, $y>0$. (6) Pour 4-gal into 3-gal until full: $(x,y)\rightarrow(x-(3-y), 3)$ if $x+y\geq3$, $x>0$. (7) Pour all of 3-gal into 4-gal: $(x,y)\rightarrow(x+y, 0)$ if $x+y\leq4$, $y>0$. (8) Pour all of 4-gal into 3-gal: $(x,y)\rightarrow(0, x+y)$ if $x+y\leq3$, $x>0$.

Step	Rule applied	4-gal jug	3-gal jug
0	Start	0	0
1	Fill 3-gal jug from pump (Rule 2)	0	3
2	Pour 3-gal into 4-gal (Rule 7)	3	0
3	Fill 3-gal jug again (Rule 2)	3	3
4	Pour 3-gal into 4-gal until full (Rule 5)	4	2
5	Empty 4-gal jug onto ground (Rule 3)	0	2
6	Pour 3-gal into 4-gal (Rule 7)	2	0

Result: Exactly 2 gallons is in the 4-gallon jug after 6 steps: $(0,0)\rightarrow(0,3)\rightarrow(3,0)\rightarrow(3,3)\rightarrow(4,2)\rightarrow(0,2)\rightarrow(2,0)$. An alternative 6-step path is $(0,0)\rightarrow(4,0)\rightarrow(1,3)\rightarrow(1,0)\rightarrow(0,1)\rightarrow(4,1)\rightarrow(2,3)$. Breadth-first search on this state space finds a 6-step solution, so it is optimal.

2. Find out how Mars Rover, analyse and explore the samples on Mars by answering the following questions.
   i. What are the percept's for this agent?

Answer: Percepts are what the rover's sensors deliver at each moment:

- Camera images (stereo/panoramic) of terrain, rocks and obstacles; LIDAR/range data.
- Spectrometer, X-ray and chemical readings of soil/rock samples (composition, moisture, organics).

- Position and motion: wheel odometry, IMU (tilt, orientation), sun sensor, GPS-like localization, wheel slip.
- Environment readings: temperature, wind, dust, radiation, light level.
- Internal status: battery/solar-panel charge, motor currents, component temperature and faults.
- Commands and updates from Earth mission control (received with a 4–24 minute delay).

ii. Characterize the operating environment.

Answer: Environment characterisation (PEAS: Environment):

- **Partially observable –** sensors see only a limited area; the terrain beyond and the sub-surface are unknown.
- **Stochastic –** wheel slip, dust storms and uncertain soil make outcomes unpredictable.
- **Sequential –** current moves affect later options (energy, position, wear).
- **Dynamic –** weather, light and temperature change while the rover is deliberating.
- **Continuous –** positions, angles and sensor values are continuous.
- **Unknown and mostly single-agent –** the map is initially unknown; other agents (orbiters, Earth control) interact only through communication.

iii. What are the actions the agent can take?

Answer: Actuators / actions available:

- Drive forward/backward, turn, steer wheels, stop, climb or avoid obstacles (re-plan route).
- Move the robotic arm; scoop, drill, grind or scrape rock and soil; place a sample in the instrument.
- Point the mast camera; take images; run the spectrometer and other analysis instruments.
- Transmit data to Earth/orbiter; receive and queue commands.
- Manage power: orient solar panels, charge, heat components, enter sleep or safe mode.

iv. How can one evaluate the performance of the agent?

Answer: Performance measure – judged on mission goals and safety:

- Number and scientific value of samples analysed (e.g., evidence of water or life).
- Area explored and distance covered successfully; accuracy of data sent to Earth.
- Safety: no collisions, falls or getting stuck; no damage to instruments.
- Efficiency: energy and time used per sample; reliable use of limited battery power.
- Mission longevity: length of time the rover remains operational.

v. What sort of agent architecture do you think is most suitable for this agent? Why?

Answer: A goal-based agent with utility-based decision making (a hybrid, layered architecture) is most suitable.

- Goal-based: the rover has explicit goals (reach a site, collect and analyse a sample) and must plan sequences of actions to reach them.
- Utility-based: it must trade off science value against risk, energy and time, so a utility function picks the best of several possible goals/routes.

- Reactive layer: simple reflex rules (stop at a cliff, tilt too high, low battery) give fast safety responses that cannot wait for a plan or for Earth.
- Learning element: improves terrain classification and navigation from experience.
- A purely simple-reflex or model-based reflex agent would be too short-sighted; a purely deliberative agent would be too slow. The layered mix handles partial observability, uncertainty and communication delay.

3. Explore the search space using search strategies and formulate the problem components for the 8-Queens problem using the following information: Place 8 queens on a chessboard such that none of the queen's attack any of the others. A configuration of 8 queens on the board as shown in the figure below, but this does not represent a solution as the queen in the first column is on the same diagonal as the queen in the last column.

Answer:

Problem formulation (incremental):

- **States:** any arrangement of n queens ($0 \leq n \leq 8$), one per column in the leftmost n columns, with no queen attacking another. This gives only 2,057 states (instead of about 1.8×10^14 for arbitrary placements of 8 queens).
- **Initial state:** empty board (no queens).
- **Actions:** add a queen to the leftmost empty column in a row where it is not attacked by any queen already placed.
- **Transition model:** returns the board with the new queen added.
- **Goal test:** 8 queens are on the board and none attacks another (no two share a row, column or diagonal).
- **Path cost:** not relevant (each step has equal cost); only the final configuration matters.

Why the given configuration fails: the queen in column 1 and the queen in column 8 lie on the same diagonal, so they attack each other. Therefore h = (number of attacking pairs) ≥ 1 and it is not a goal state.

Search strategies:

- **Breadth-first search:** complete, but must store a huge number of partial boards at each level – memory-heavy.
- **Depth-first search with backtracking (best uninformed choice):** place queens column by column; when a column has no safe row, backtrack to the previous column and try the next row. Memory use is only O(8), and it finds a solution quickly because every solution is at depth 8.
- **Local search (e.g., hill climbing / min-conflicts):** start from a full board like the one in the figure and move a queen in its column to reduce conflicts; works well when the full-state formulation is used.

Solution found by backtracking: queens in columns 1–8 are placed in rows 1, 5, 8, 6, 3, 7, 2, 4, i.e. (c1,r1), (c2,r5), (c3,r8), (c4,r6), (c5,r3), (c6,r7), (c7,r2), (c8,r4). No two share a row, column or diagonal, so the goal test is satisfied. (In this solution columns 1 and 8 are in rows 1 and 4, no longer on a common diagonal, which fixes the fault in the figure.)

4. A customer wants to travel from the one location to another location using OLA Cab booking mobile application. The customer can select the any of the cab type as mini, micro, sedan, shared, prime and so on based on his comfort to reach the destination. Identify what type of problem-solving agent can be used and write the pseudocode for the above problem.

Answer:

Type of agent: A goal-based problem-solving agent that uses utility-based selection. The goal is to reach the destination; the agent searches over the possible actions (cab type and route) and uses a utility function (fare, time, comfort, availability) to choose the best one. The environment is fully observable at booking time (the app knows locations, prices and ETA), sequential and dynamic (traffic and driver availability change).

Problem formulation: State = (user location, destination, cab type, status). Initial state = user at pickup location. Actions = select cab type (Mini, Micro, Sedan, Shared, Prime…), confirm booking, drive along a route. Goal test = customer reached destination. Path cost = fare + travel time (and optionally waiting time).

Pseudocode:

```
function OLA-AGENT(percept) returns an action
  persistent: seq (action sequence, initially empty)
              state, goal, problem

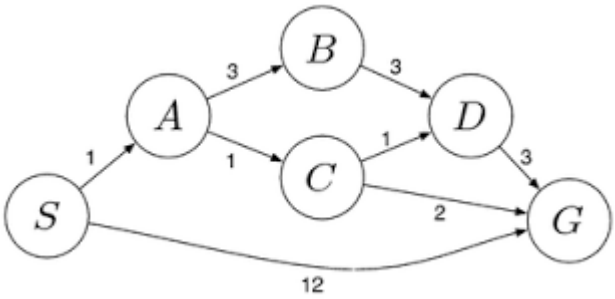
  state <- UPDATE-STATE(state, percept)
  if seq is empty then
      goal    <- FORMULATE-GOAL(state)
      problem <- FORMULATE-PROBLEM(state, goal)
      seq     <- SEARCH(problem)
      if seq = failure then return NO-CAB-AVAILABLE
  action <- FIRST(seq)
  seq    <- REST(seq)
  return action

function SEARCH(problem)
  input : pickup, drop, cabTypes = {Mini, Micro, Sedan, Shared, Prime}
  best <- null; bestUtility <- -infinity
  for each cab in cabTypes do
      if AVAILABLE(cab, pickup) then
          fare  <- ESTIMATE-FARE(cab, pickup, drop)
          time  <- ESTIMATE-ETA(cab, pickup, drop)
          route <- A*-SEARCH(pickup, drop)
          U <- w1*COMFORT(cab) - w2*fare - w3*time
          if U > bestUtility then
              best <- (cab, route); bestUtility <- U
  if best = null then return failure
  return [SELECT(best.cab), CONFIRM-BOOKING, PICKUP,
          FOLLOW(best.route), DROP, PAY]
```

Explanation: the customer's preferences (budget, comfort, speed) are captured in the weights w1, w2, w3. The agent compares all available cab types, selects the one with the highest utility, finds the route with A* search and executes the actions until the goal (reaching the destination) is achieved.

5. A logistics company operates a delivery network where packages must be transported between warehouses at minimum cost. Each route between warehouses has an associated travel cost (fuel + time). The company wants to determine the least-cost path from the starting warehouse **S** to the destination warehouse G using the Uniform Cost Search (UCS) algorithm.

The delivery network is represented as a weighted graph:



Answer (Uniform Cost Search):

Graph edges (from the figure): S–A = 1, S–G = 12, A–B = 3, A–C = 1, B–D = 3, C–D = 1, C–G = 2, D–G = 3. UCS keeps a priority queue ordered by path cost g(n) and always expands the lowest-cost node.

Step	Node expanded (g)	Frontier after expansion (node : cost)
1	S (0)	A:1, G:12
2	A (1)	C:2, B:4, G:12
3	C (2)	D:3, B:4, G:4, G:12 (G via C = 2+2 = 4 replaces 12)
4	D (3)	B:4, G:4 (G via D = 3+3 = 6, not better)
5	B (4)	G:4 (D via B = 7, not better)
6	G (4)	Goal reached – stop

Order of expansion: S → A → C → D → B → G.

Least-cost path: S → A → C → G with total cost = 1 + 1 + 2 = 4.

Comparison: The direct edge S → G costs 12, and S → A → C → D → G costs 1+1+1+3 = 6, and S → A → B → D → G costs 10. Hence the least-cost route is S → A → C → G (cost 4). UCS is complete and optimal because all edge costs are positive.

Interpretation: The company should route the package from warehouse S through A and C to reach G, which minimises the combined fuel and time cost (4 cost units), compared with 12 units for the direct route.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem
Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method

Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem